

# 1 Learning Goals

This exercise's learning goals are to help students understand and apply foundational concepts of RESTful API design in a practical setting. Students will learn to define resources and their associated operations, focusing on adhering to REST principles such as consistency, usability, and scalability.

Students will enhance their ability to design robust APIs by creating state diagrams, identifying allowed and disallowed states, and determining the appropriate use of headers and body in responses. Additionally, the exercise fosters an understanding client-side state management and collaboration through group discussions, preparing students for real-world software development challenges.

# 2 Foundations

The university plans to develop a room booking system (RBS) to manage conference rooms. This application aims to streamline the reservation process, ensuring efficient scheduling and availability tracking.

As part of this initiative, your task is to design a RESTful API for RBS. The API should follow REST principles, providing clear and consistent endpoints for creating, retrieving, updating, and deleting booking records.

It must include functionality to check room availability, manage user authentication, and handle potential conflicts during reservations. The design should prioritize usability, scalability, and maintainability to meet the university's long-term needs.

This is the list of requirements:

- Users can ask for a list of conference rooms.
- Users can ask for a list of free conference rooms.
- Users can not create or delete conference rooms
- Users can not update the attributes of existing conference rooms.
- Users can book a room by giving a start and end time and a description.
- A booking always has an owner (identified by a `common name`).

- A booking can be modified and deleted.
- Users can ask for all bookings for a single room.
- Users can ask for all bookings on a specific date for a single room.

### 3 Methodology

1. Form groups of three to four students.
2. Define the resources. It is not necessary to define the attributes in detail.
3. Define the URLs and CRUD operations.
4. Make clear what states (combinations of URL and HTTP verbs) are **not** allowed.
5. Draw a diagram that contains boxes as states and arrows as transitions.
6. Annotate the transitions to define if the URL is given as a response in the **header** or the **body**.
7. Discuss in the group the difference between header and body links. When do you use what?
8. Discuss how to develop a REST client. The main question is: What is the state on the client side, and which class maintains it?

### 4 Deliverables

Please take a photo of your state diagram and upload it to the Moodle activity.